

BUILDING REST APIs WITH DJANGO

PROGRAM BRIEF | FIVE-DAY HANDS-ON PROGRAM

TASK 01

FINAL
DRAFT
FOR
APPROVAL

ORGANIZATION	TRACK	PROGRAM DATES	DOCUMENT
Tuwaiq Club at KFUPM	Modern Software & Cloud	October 11-15, 2026	Version 1.0 Sep 27, 2026

01 PURPOSE AND FINAL OUTCOME

PURPOSE

Define the audience, learning scope, project requirements, daily milestones, and verifiable evidence of success for a five-day Django REST API program.

FINAL OUTCOME

By the end of Day 5, participants will be able to design, build, test, document, and deploy a database-backed REST API using Django REST Framework, including authentication, validated CRUD operations, and per-user permissions.

02 AUDIENCE AND ENTRY REQUIREMENTS

WHO IT IS FOR

KFUPM university students and beginner developers who can write small Python programs and want practical backend development experience. Prior experience with Django, REST APIs, databases, or cloud deployment is **not required**.

REQUIRED BEFORE DAY 1

- Python variables, functions, conditionals, loops, lists, and dictionaries
- Basic classes, objects, and exception handling
- Terminal navigation plus `pip` and virtual environments
- Basic Git/GitHub: clone, add, commit, push, and pull
- A laptop with Python, Git, a code editor, and a GitHub account

03 WHAT PARTICIPANTS WILL LEARN

- Explain how HTTP methods, URLs, JSON, status codes, and request-response cycles work.
- Model relational data with Django ORM, migrations, and database-backed persistence.
- Build serializers, views/viewsets, routes, and CRUD endpoints with Django REST Framework.
- Apply token authentication, ownership rules, object permissions, and input validation.
- Write automated API tests and diagnose failures using clear test feedback.
- Configure, document, deploy, and verify an API in a production environment.

PRACTICAL PROJECT: PERSONAL TASK MANAGEMENT REST API

Starting on Day 1, each participant builds one API through five connected increments. Authenticated users can create and manage their own tasks while the system validates input and prevents access to other users' data. The project ends as a tested, documented, deployed service. **No frontend is required.**

04 MINIMUM PROJECT REQUIREMENTS

AREA	REQUIRED FOR COMPLETION	ACCEPTANCE CHECK
Data model	Task fields: id, title, description, status, due_date, owner, created_at, and updated_at. Status choices: TODO, IN_PROGRESS, DONE.	Migrations apply successfully and data persists after restart/redeploy.
CRUD API	Authenticated users can create, list, retrieve, fully/partially update, and delete tasks.	Every operation returns an appropriate JSON response and HTTP status code.
Authentication	Django REST Framework token authentication. Tokens are issued through a login endpoint. Test users may be created through Django admin or a documented command.	Protected task endpoints reject unauthenticated requests with 401.
Ownership	The API assigns owner from the authenticated user. Users can list, retrieve, update, and delete only their own tasks.	A second user cannot view or modify the first user's tasks.
Validation	Title is required and nonblank; status must use an allowed value; due date cannot be earlier than today when a task is created; owner is read-only.	Invalid input returns 400 with clear field-level error messages.
Testing	At least 8 automated tests using DRF APITestCase or equivalent, covering authentication, CRUD, ownership/data isolation, and validation.	The documented test command completes with all tests passing.
Documentation	README includes setup, environment variables, migrations, test command, authentication, endpoint examples, deployed URL, and project decisions.	A new reviewer can run and use the project from the README.
Deployment	Public HTTPS deployment on the program-approved cloud platform, with production settings and persistent database configuration.	The live URL supports the required demonstration flow and retains data.
Source control	GitHub repository visible to reviewers, meaningful commit history, dependency file, sample environment file, and no committed secrets.	Repository can be cloned; .env is ignored; setup is reproducible.

05 REQUIRED API CONTRACT

METHOD	PATH	PURPOSE	AUTH
POST	/api/auth/token/	Exchange valid username/password for an API token	No
GET	/api/tasks/	List only the authenticated user's tasks	Yes
POST	/api/tasks/	Create a task owned by the authenticated user	Yes
GET	/api/tasks/{id}/	Retrieve one owned task	Yes
PUT/PATCH	/api/tasks/{id}/	Replace or partially update one owned task	Yes
DELETE	/api/tasks/{id}/	Delete one owned task	Yes

OPTIONAL EXTENSIONS

- Filtering, search, ordering, or pagination
- Priority, tags, categories, or completion statistics
- JWT authentication or user registration
- OpenAPI documentation and Swagger UI
- CI, Docker, throttling, or expanded test coverage

EXPLICITLY OUT OF SCOPE

- Frontend, mobile application, or visual dashboard
- Email verification, password reset, or social login
- Complex roles, teams, sharing, or notifications
- Microservices or production operations beyond one deployment
- Optional items as conditions for passing

Scope rule: Optional work begins only after all minimum requirements pass locally. Optional features never block program completion.

06 FIVE CONNECTED DAILY MILESTONES

Delivery baseline: five approximately three-hour guided sessions (about 15 contact hours), supported by up to one hour of optional independent work per day. The leads should adjust facilitation if the confirmed contact time differs.

DAY 1	FOUNDATION	Oct 11
LEARN	HTTP/JSON orientation; environment setup; Django and DRF structure; ORM, models, and migrations; Git workflow.	BUILD Create the repository and virtual environment; start the Django project and tasks app; add dependencies; implement the Task model; run migrations; create users and sample tasks through admin.
EXIT CHECK	Repository is pushed. Server starts locally. Admin shows saved Task records after restart.	
DAY 2	CORE API	Oct 12
LEARN	Serializers; validation basics; generic views or viewsets; routers and URLs; CRUD semantics; HTTP status codes.	BUILD Implement serializer, viewset, router, and the required task endpoints. Exercise create, list, retrieve, update, partial update, and delete using an API client.
EXIT CHECK	All CRUD operations work locally with valid JSON and appropriate status codes.	
DAY 3	SECURITY	Oct 13
LEARN	Token authentication; authenticated requests; queryset filtering; automatic ownership; object permissions; clear validation errors.	BUILD Add the token endpoint, require authentication, assign owner from request.user, filter lists by user, block cross-user access, and implement the required validation rules.
EXIT CHECK	User A cannot view or modify User B's tasks. Invalid data is rejected with field-level errors.	
DAY 4	QUALITY AND DELIVERY	Oct 14
LEARN	API testing; test data and isolation; README writing; environment variables; production settings; deployment workflow.	BUILD Write at least 8 automated tests; complete the README; add sample environment configuration; prepare production settings and dependency files; perform a deployment rehearsal.
EXIT CHECK	All tests pass. A fresh clone can be set up from the README. Deployment configuration is ready.	
DAY 5	PUBLISH AND DEMONSTRATE	Oct 15
LEARN	Cloud release; migrations; environment configuration; production troubleshooting; smoke testing; concise technical demonstrations.	BUILD Deploy the API and database; run production migrations; verify the live endpoints; correct release issues; submit evidence; complete the final demonstration.
EXIT CHECK	Public HTTPS URL works, data persists, all required evidence is submitted, and the live demo passes.	

PACING AND DELIVERY SAFEGUARDS

- The instructor maintains a known-good reference branch for the end of each day.
- Participants who miss an exit check start the next session from the reference checkpoint, preserving Day 5 completion.
- Cloud accounts and the approved platform are verified before Day 4; deployment is rehearsed on Day 4, not left entirely to Day 5.
- Stretch features are taught only after the minimum project works and all required tests pass.

07 FINAL EVIDENCE OF COMPLETION

SUBMISSION PACKAGE

- GitHub repository URL accessible to reviewers
- Public HTTPS deployment URL
- README with setup, usage, endpoint, test, and deployment instructions
- Passing automated test output for at least 8 tests
- No exposed credentials or secrets

REQUIRED LIVE DEMONSTRATION

- Obtain a token and authenticate
- Create and list personal tasks
- Update one task and show validation failure
- Prove cross-user data isolation
- Delete a task and confirm the live service persists data

08 DEFINITION OF DONE AND ASSESSMENT

COMPLETION CHECKLIST - EVERY ITEM MUST PASS

- All required Task fields and CRUD endpoints are implemented.
- Protected endpoints require authentication and enforce per-user ownership.
- Required validation returns correct error responses.
- At least 8 automated tests pass with no required tests skipped.
- The repository is reproducible, documented, and free of committed secrets.
- The deployed API is publicly reachable over HTTPS and uses persistent storage.
- The participant completes the required live demonstration without instructor code changes.

ASSESSMENT AREA	WEIGHT	PASS STANDARD
Core API and data model	25%	Required fields, persistence, and complete CRUD behavior
Authentication and ownership	25%	Protected endpoints and successful cross-user isolation check
Validation and API behavior	15%	Required rules, clear errors, and appropriate status codes
Automated tests	15%	At least 8 meaningful tests; all required tests pass
Documentation and repository	10%	Reproducible setup, useful README, clean Git history, no secrets
Deployment and demonstration	10%	Working HTTPS deployment and complete live demonstration

Passing rule: Every Definition of Done item is mandatory. The rubric supports consistent feedback; it does not allow a participant to compensate for a missing mandatory item with optional features.

09 LEAD REVIEW AND APPROVAL

REVIEWER	REVIEW FOCUS	DECISION	NAME / SIGNATURE / DATE
Curriculum Lead	Audience fit, learning progression, workload, outcomes, and assessment alignment	Approved Changes requested	_____ _____
Technical Lead	Architecture, API contract, security, testability, deployment feasibility, and technical accuracy	Approved Changes requested	_____ _____

DOCUMENT STATUS

Ready for Curriculum Lead and Technical Lead review. Scope changes after approval should be recorded in a new document version.